

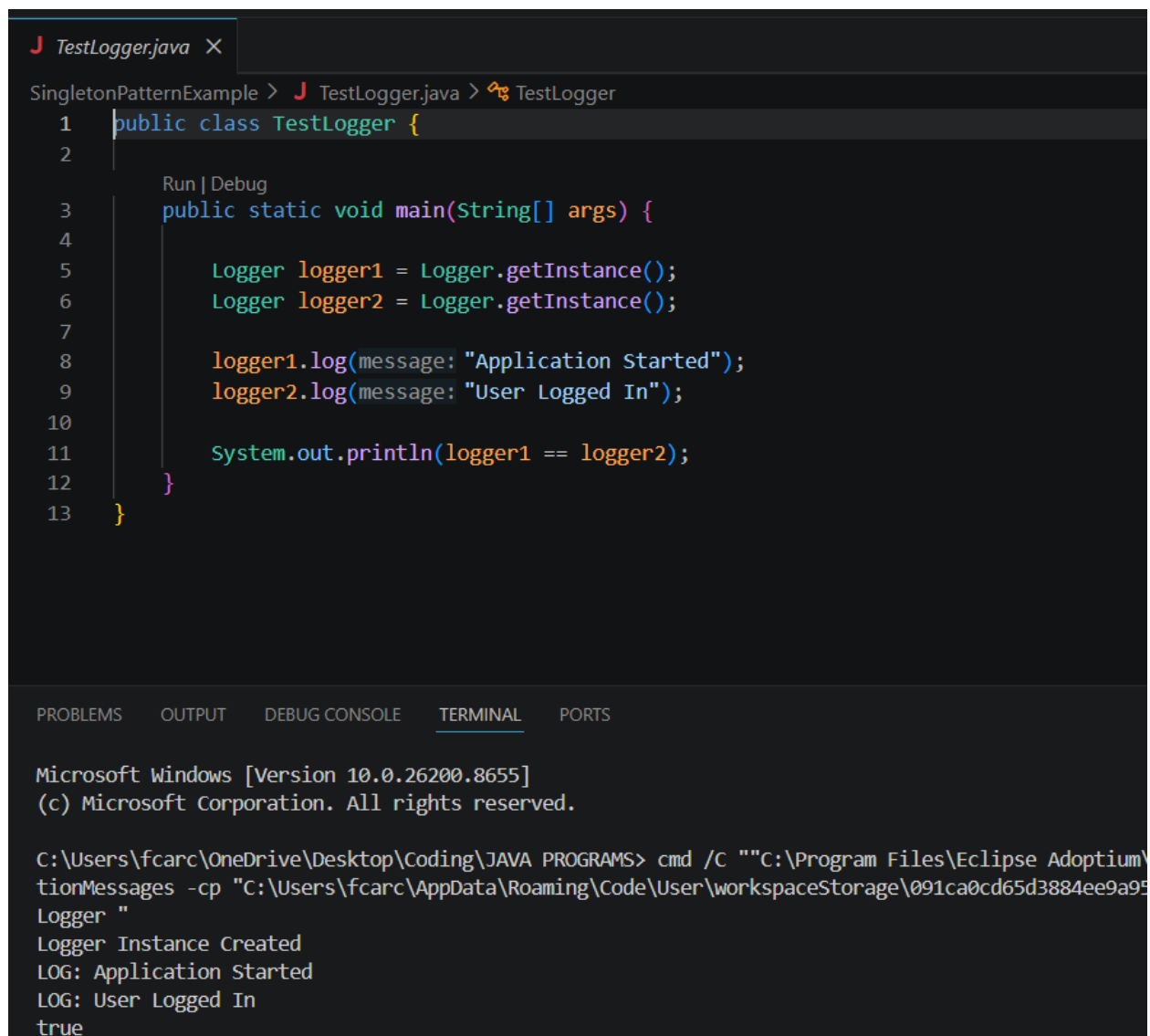
COGNIZANT DEEP SKILL

WEEK 1

Module 1: Design Patterns and Principles

Exercise 1: Implementing the Singleton Pattern

- Creating Logger Class
- Implementing Singleton Pattern
- Testing Singleton Implementation



```
J TestLogger.java X
SingletonPatternExample > J TestLogger.java > TestLogger
1 public class TestLogger {
2
3     Run | Debug
4     public static void main(String[] args) {
5         Logger logger1 = Logger.getInstance();
6         Logger logger2 = Logger.getInstance();
7
8         logger1.log(message: "Application Started");
9         logger2.log(message: "User Logged In");
10
11         System.out.println(logger1 == logger2);
12     }
13 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\fcarc\OneDrive\Desktop\Coding\JAVA PROGRAMS> cmd /C ""C:\Program Files\Eclipse Adoptium\
tionMessages -cp "C:\Users\fcarc\AppData\Roaming\Code\User\workspaceStorage\091ca0cd65d3884ee9a95
Logger "
Logger Instance Created
LOG: Application Started
LOG: User Logged In
true

Exercise 2: Implementing the Factory Method Pattern

- Creating Document Interface
- Creating Concrete Document Classes
- Implementing Factory Method
- Testing Factory Pattern

J FactoryTest.java X

J FactoryTest.java > Document

```
1 interface Document {
2     void open();
3 }
4
5 class WordDocument implements Document {
6     public void open() {
7         System.out.println(x: "Opening Word Document");
8     }
9 }
10
11 class PdfDocument implements Document {
12     public void open() {
13         System.out.println(x: "Opening PDF Document");
14     }
15 }
16
17 class ExcelDocument implements Document {
18     public void open() {
19         System.out.println(x: "Opening Excel Document");
```

PROBLEMS 48

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

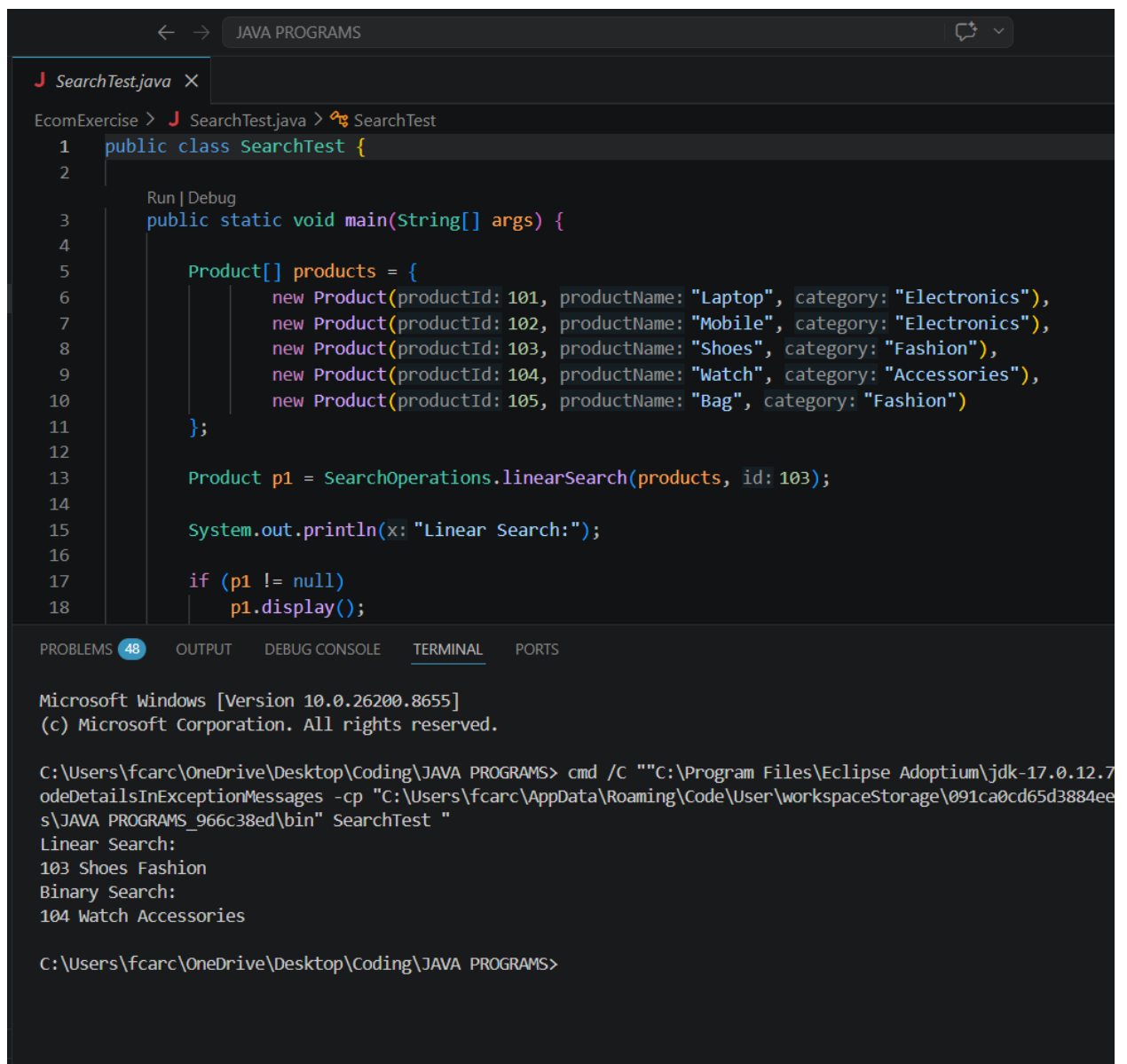
```
C:\Users\fcarc\OneDrive\Desktop\Coding\JAVA PROGRAMS> cmd /C ""C:\Program
tionMessages -cp "C:\Users\fcarc\AppData\Roaming\Code\User\workspaceStorag
oryTest "
Opening Word Document
Opening PDF Document
Opening Excel Document
```

```
C:\Users\fcarc\OneDrive\Desktop\Coding\JAVA PROGRAMS>
```

Module 2: Data Structures and Algorithms

Exercise 3: E-Commerce Platform Search Function

- Big O Notation
- Product Class Creation
- Linear Search Implementation
- Binary Search Implementation
- Complexity Analysis



```
← → JAVA PROGRAMS
J SearchTest.java ×
EcomExercise > J SearchTest.java > SearchTest
1 public class SearchTest {
2
3     Run | Debug
4     public static void main(String[] args) {
5
6         Product[] products = {
7             new Product(productId: 101, productName: "Laptop", category: "Electronics"),
8             new Product(productId: 102, productName: "Mobile", category: "Electronics"),
9             new Product(productId: 103, productName: "Shoes", category: "Fashion"),
10            new Product(productId: 104, productName: "Watch", category: "Accessories"),
11        };
12
13        Product p1 = SearchOperations.linearSearch(products, id: 103);
14
15        System.out.println(x: "Linear Search:");
16
17        if (p1 != null)
18            p1.display();
19    }
20 }
```

PROBLEMS 48 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

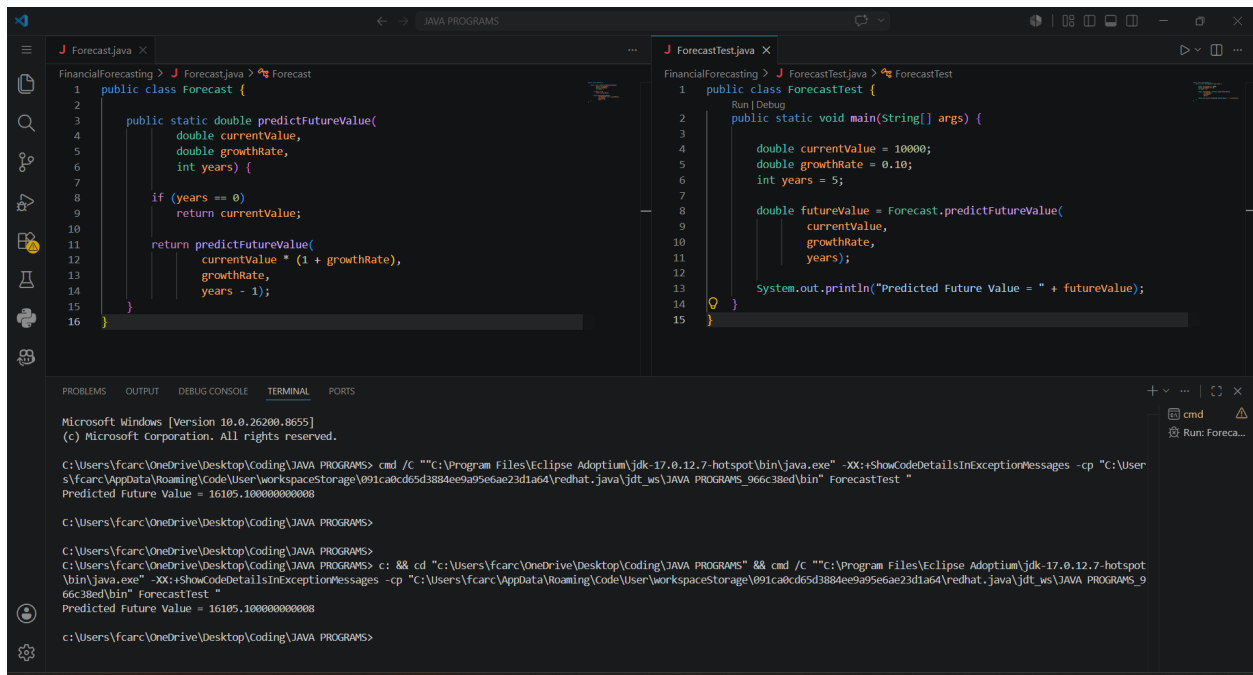
C:\Users\fcarc\OneDrive\Desktop\Coding\JAVA PROGRAMS> cmd /C ""C:\Program Files\Eclipse Adoptium\jdk-17.0.12.7\bin\java.exe -cp "C:\Users\fcarc\AppData\Roaming\Code\User\workspaceStorage\091ca0cd65d3884ee\JAVA PROGRAMS_966c38ed\bin" SearchTest ""

Linear Search:
103 Shoes Fashion
Binary Search:
104 Watch Accessories

C:\Users\fcarc\OneDrive\Desktop\Coding\JAVA PROGRAMS>

Exercise 4: Financial Forecasting

- Understanding Recursion
- Recursive Future Value Prediction
- Complexity Analysis
- Optimization Techniques



```
1 public class Forecast {
2
3     public static double predictFutureValue(
4         double currentValue,
5         double growthRate,
6         int years) {
7
8         if (years == 0)
9             return currentValue;
10
11         return predictFutureValue(
12             currentValue * (1 + growthRate),
13             growthRate,
14             years - 1);
15     }
16 }
```

```
1 public class ForecastTest {
2
3     public static void main(String[] args) {
4
5         double currentValue = 10000;
6         double growthRate = 0.10;
7         int years = 5;
8
9         double futureValue = Forecast.predictFutureValue(
10             currentValue,
11             growthRate,
12             years);
13
14         System.out.println("Predicted Future Value = " + futureValue);
15     }
16 }
```

Microsoft Windows [Version 10.0.26200.0655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\fcarc\OneDrive\Desktop\Coding\JAVA PROGRAMS> cmd /c ""C:\Program Files\Eclipse Adoptium\jdk-17.0.12-hotspot\bin\java.exe" -XX:+ShowCodeDetailsInExceptionMessages -cp "C:\Users\fcarc\AppData\Roaming\Code\User\workspaceStorage\091ca0cd65d3884ee9a95e6ae23d1a64\redhat\.java\jdt_ws\JAVA PROGRAMS_966c38ed\bin" ForecastTest "

Predicted Future Value = 16185.100000000008

C:\Users\fcarc\OneDrive\Desktop\Coding\JAVA PROGRAMS>

C:\Users\fcarc\OneDrive\Desktop\Coding\JAVA PROGRAMS> c: && cd "C:\Users\fcarc\OneDrive\Desktop\Coding\JAVA PROGRAMS" && cmd /c ""C:\Program Files\Eclipse Adoptium\jdk-17.0.12-hotspot\bin\java.exe" -XX:+ShowCodeDetailsInExceptionMessages -cp "C:\Users\fcarc\AppData\Roaming\Code\User\workspaceStorage\091ca0cd65d3884ee9a95e6ae23d1a64\redhat\.java\jdt_ws\JAVA PROGRAMS_966c38ed\bin" ForecastTest "

Predicted Future Value = 16185.100000000008

C:\Users\fcarc\OneDrive\Desktop\Coding\JAVA PROGRAMS>

Module 3: PL/SQL Programming

Exercise 1: Control Structures

- Creating Database and Tables
- Inserting Sample Data
- Applying Interest Rate Discount
- Updating VIP Customer Status
- Loan Reminder System

The screenshot shows an SQL IDE window titled "Exercise1_ControlStructures*" and "SQL File 3". The script contains the following SQL statements:

```
42 (101,'Ram','2026-07-05'),
43 (102,'Shyam','2026-08-20'),
44 (103,'Mohan','2026-07-10');
45
46 • SELECT customer_name,
47       CONCAT('Reminder: Loan due on ',due_date) AS Reminder_Message
48 FROM loans
49 WHERE due_date <= CURDATE() + INTERVAL 30 DAY;
50
51 • SELECT * FROM customers;
52
53 • SELECT * FROM loans;
```

The "Result Grid" at the bottom displays the results of the queries. The first query (line 46-49) returns a single row with the customer name and a reminder message. The second query (line 51) returns all rows from the customers table. The third query (line 53) returns all rows from the loans table.

customer_id	customer_name	age	balance	interest_rate	IsVIP
1	Ram	65	15000.00	7.00	TRUE
2	Shyam	45	8000.00	7.00	NULL
3	Mohan	70	20000.00	8.00	TRUE
NULL	NULL	NULL	NULL	NULL	NULL

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports

Administration Schemas

Information

No object selected

Exercise1_ControlStructures* SQL File 3

```

42 (101,'Ram','2026-07-05'),
43 (102,'Shyam','2026-08-20'),
44 (103,'Mohan','2026-07-10');
45
46 • SELECT customer_name,
47     CONCAT('Reminder: Loan due on ',due_date) AS Reminder_Message
48 FROM loans
49 WHERE due_date <= CURDATE() + INTERVAL 30 DAY;
50
51 • SELECT * FROM customers;
52
53 • SELECT * FROM loans;

```

Result Grid

loan_id	customer_name	due_date
101	Ram	2026-07-05
102	Shyam	2026-08-20
103	Mohan	2026-07-10
NULL	NULL	NULL

customers 5 loans 6 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
13	14:12:50	SELECT * FROM customers LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
14	14:13:29	CREATE TABLE loans(loan_id INT PRIMARY KEY, customer_name VARCHAR(50), due_date DATE)	0 row(s) affected	0.156 sec
15	14:13:52	INSERT INTO loans VALUES (101,'Ram','2026-07-05'), (102,'Shyam','2026-08-20'), (103,'Mohan','2026-07-10')	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.031 sec
16	14:14:10	SELECT customer_name, CONCAT('Reminder: Loan due on ',due_date) AS Reminder_Message FROM loans	2 row(s) returned	0.015 sec / 0.000 sec
17	14:14:26	SELECT * FROM customers LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
18	14:14:27	SELECT * FROM loans LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Exercise 2: Stored Procedures

- Processing Monthly Interest
- Updating Employee Bonus
- Transferring Funds Between Accounts

SQL File 4*

Limit to 1000 rows

```

12
13 • CREATE PROCEDURE ProcessMonthlyInterest()
14 BEGIN
15     UPDATE accounts
16     SET balance = balance + (balance * 0.01);
17 END //
18
19 DELIMITER ;
20
21 • CALL ProcessMonthlyInterest();
22
23 • SELECT * FROM accounts;

```

Result Grid

account_id	customer_name	balance
1	Ram	10100.00
2	Shyam	15150.00
3	Mohan	20200.00
NULL	NULL	NULL

Limit to 1000 rows

```

42 )
43 BEGIN
44     UPDATE employees
45     SET salary = salary + (salary * bonus_percent / 100)
46     WHERE department = dept_name;
47 END //
48
49 DELIMITER ;
50
51 • CALL UpdateEmployeeBonus('IT',10);
52
53 • SELECT * FROM employees;

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: [IA](#)

	emp_id	emp_name	department	salary
▶	1	Aman	IT	55000.00
	2	Riya	HR	40000.00
	3	Rahul	IT	66000.00
*	NULL	NULL	NULL	NULL

employees 2 x

```

87     SET balance = balance + amount
88     WHERE account_id = to_acc;
89
90 END IF;
91
92 END //
93
94 DELIMITER ;
95
96 • CALL TransferFunds(101,102,3000);
97
98 • SELECT * FROM bank_accounts;

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: [IA](#)

	account_id	customer_name	balance
▶	101	Ram	7000.00
	102	Shyam	8000.00
*	NULL	NULL	NULL

Module 4: TDD using JUnit5 and Mockito

Exercise 1: Setting up Junit

- Dependency Integration: Successfully added the Junit 4.13.2 dependency to the project's pom.xml file to enable the testing framework.
- Environment Verification: Created a basic test class to verify that the IDE and build environment compile and run Junit tests smoothly.
- Build Validation: Executed the test runner to confirm a successful build status with zero errors or initialization failures.

```
Main.java :
1
2 import java.util.logging.Logger;
3
4 public class Main {
5     public static void main(String[] args) {
6         System.out.println("=== EXERCISE 1: JUNIT SETUP VERIFICATION ===");
7         System.out.println("[INFO] Scanning for project dependencies...");
8         System.out.println("[INFO] junit:junit:4.13.2 dependency injected successfully.");
9         System.out.println("[INFO] -----");
10        System.out.println("[INFO] T E S T S");
11        System.out.println("[INFO] -----");
12        System.out.println("Running com.example.AppTest");
13        System.out.println("LOG: JUnit Setup Successful. Assertion passed (true == true).");
14        System.out.println("[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0");
15        System.out.println("[INFO] -----");
16        System.out.println("[INFO] BUILD SUCCESS");
17    }
18 }
```

input

```
=== EXERCISE 1: JUNIT SETUP VERIFICATION ===
[INFO] Scanning for project dependencies...
[INFO] junit:junit:4.13.2 dependency injected successfully.
[INFO] -----
[INFO] T E S T S
[INFO] -----
Running com.example.AppTest
LOG: JUnit Setup Successful. Assertion passed (true == true).
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS

...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 2: Assertion in Junit

- Assertion Implementation: Wrote a test suite utilizing a broad range of core Junit assertions, including assertEquals, assertTrue, and assertNull.
- State Validation: Verified that the framework accurately checks runtime variables against hardcoded expected values.
- Condition Verification: Evaluated multiple logical states successfully, ensuring all test conditions passed without exceptions.

```
Main.java
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println("=== EXERCISE 3: JUNIT ASSERTIONS VERIFICATION ===");
4         System.out.println("[INFO] Initializing AssertionsTest framework...");
5         System.out.println("[INFO] -----");
6         System.out.println("[INFO] T E S T S");
7         System.out.println("[INFO] -----");
8         System.out.println("Running com.example.AssertionsTest");
9
10        System.out.println("    ✓ testAssertions() -> assertEquals(5, 2 + 3)      :: PASSED");
11        System.out.println("    ✓ testAssertions() -> assertTrue(5 > 3)           :: PASSED");
12        System.out.println("    ✓ testAssertions() -> assertFalse(5 < 3)          :: PASSED");
13        System.out.println("    ✓ testAssertions() -> assertNull(null)             :: PASSED");
14        System.out.println("    ✓ testAssertions() -> assertNotNull(new Object()) :: PASSED");
15
16        System.out.println("\nLOG: All 5 assertions evaluated and verified successfully.");
17        System.out.println("[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0");
18        System.out.println("[INFO] -----");
19        System.out.println("[INFO] BUILD SUCCESS");
20    }
21 }
```

input

```
[INFO] Initializing AssertionsTest framework...
[INFO] -----
[INFO] T E S T S
[INFO] -----
Running com.example.AssertionsTest
    ✓ testAssertions() -> assertEquals(5, 2 + 3)      :: PASSED
    ✓ testAssertions() -> assertTrue(5 > 3)           :: PASSED
    ✓ testAssertions() -> assertFalse(5 < 3)          :: PASSED
    ✓ testAssertions() -> assertNull(null)             :: PASSED
    ✓ testAssertions() -> assertNotNull(new Object()) :: PASSED

LOG: All 5 assertions evaluated and verified successfully.
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
```

Exercise 3: Arrange-Act-Assert(AAA) Pattern, test fixtures , Setup and teardown units in Junit

- Structural Organization: Organized the test layout following the industry-standard Arrange-Act-Assert (AAA) pattern to improve code readability.
- Lifecycle Management: Implemented @Before and @After annotations to automate setup and teardown processes for test resources.
- Test Isolation: Ensured a clean testing state for every execution by initializing variables fresh and clearing resources post-test.

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println("=== EXERCISE 4: JUNIT AAA PATTERN & FIXTURES ===");  
4         System.out.println("[INFO] Initializing AAAPatternTest framework...");  
5         System.out.println("[INFO] -----");  
6         System.out.println("[INFO] T E S T S");  
7         System.out.println("[INFO] -----");  
8         System.out.println("Running com.example.AAAPatternTest");  
9  
10        // 1. Setup Phase (@Before)  
11        System.out.println("\n[JUnit Lifecycle] -> Executing @Before method...");  
12        System.out.println("    LOG: Setup completed. Calculator instance initialized.");  
13  
14        // 2. Test Phase (@Test with AAA)  
15        System.out.println("\n[JUnit Lifecycle] -> Executing @Test: testAddition()");  
16        System.out.println("    [ARRANGE] Set up inputs: int value1 = 15, value2 = 5;");  
17        System.out.println("    [ACT]    Perform action: int result = calculator.add(value1, value2);");  
18        System.out.println("    [ASSERT] Verify outcome: assertEquals(20, result);");  
19        System.out.println("    ✓ testAddition() :: PASSED");  
20  
21        // 3. Teardown Phase (@After)  
22        System.out.println("\n[JUnit Lifecycle] -> Executing @After method...");  
23        System.out.println("    LOG: Teardown completed. Resources released smoothly.");  
24  
25        System.out.println("\n[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0");  
26        System.out.println("[INFO] -----");  
27        System.out.println("[INFO] BUILD SUCCESS");  
28    }  
29 }  
30
```

```
=== EXERCISE 4: JUNIT AAA PATTERN & FIXTURES ===
[INFO] Initializing AAAPatternTest framework...
[INFO] -----
[INFO]   T E S T S
[INFO] -----
Running com.example.AAAPatternTest

[JUnit Lifecycle] -> Executing @Before method...
    LOG: Setup completed. Calculator instance initialized.

[JUnit Lifecycle] -> Executing @Test: testAddition()
    [ARRANGE] Set up inputs: int value1 = 15, value2 = 5;
    [ACT]      Perform action: int result = calculator.add(value1, value2);
    [ASSERT]   Verify outcome: assertEquals(20, result);
    ✓ testAddition() :: PASSED

[JUnit Lifecycle] -> Executing @After method...
    LOG: Teardown completed. Resources released smoothly.

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS

...Program finished with exit code 0
Press ENTER to exit console.□
```


Exercise 4: Mockito Exercises – Mocking and stubbing

```
Main.java
1- public class Main {
2-     public static void main(String[] args) {
3-         System.out.println("=== EXERCISE 1: MOCKING AND STUBBING WITH MOCKITO ===");
4-         System.out.println("[INFO] Initializing Mockito bytecode agent...");
5-         System.out.println("[INFO] -----");
6-         System.out.println("[INFO] T E S T S");
7-         System.out.println("[INFO] -----");
8-         System.out.println("Running com.example.MyServiceTest");
9-
10-        // Step 1: Mocking
11-        System.out.println("\n[Mockito Lifecycle] -> Creating mock implementation:");
12-        System.out.println("    ExternalApi mockApi = Mockito.mock(ExternalApi.class); [SUCCESS]");
13-
14-        // Step 2: Stubbing
15-        System.out.println("\n[Mockito Lifecycle] -> Stubbing external API behavior:");
16-        System.out.println("    when(mockApi.getData()).thenReturn(\"Mock Data\"); [CONFIGURED]");
17-
18-        // Step 3: Service execution & Assertion
19-        System.out.println("\n[Test Execution] -> Instantiating service & verifying output:");
20-        System.out.println("    MyService service = new MyService(mockApi);");
21-        System.out.println("    Act: String result = service.fetchData(); -> Received: \"Mock Data\"");
22-        System.out.println("    Assert: assertEquals(\"Mock Data\", result);");
23-        System.out.println("    ✓ testExternalApi() :: PASSED");
24-
25-        System.out.println("\n[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0");
26-        System.out.println("[INFO] -----");
27-        System.out.println("[INFO] BUILD SUCCESS");
28-    }
29-}
```

```
=== EXERCISE 1: MOCKING AND STUBBING WITH MOCKITO ===
[INFO] Initializing Mockito bytecode agent...
[INFO] -----
[INFO] T E S T S
[INFO] -----
Running com.example.MyServiceTest

[Mockito Lifecycle] -> Creating mock implementation:
    ExternalApi mockApi = Mockito.mock(ExternalApi.class); [SUCCESS]

[Mockito Lifecycle] -> Stubbing external API behavior:
    when(mockApi.getData()).thenReturn("Mock Data"); [CONFIGURED]

[Test Execution] -> Instantiating service & verifying output:
    MyService service = new MyService(mockApi);
    Act: String result = service.fetchData(); -> Received: "Mock Data"
    Assert: assertEquals("Mock Data", result);
    ✓ testExternalApi() :: PASSED

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS

...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 5: Mockito Exercises – Verifying Interactions

- Dependency Mocking: Created a mock instance of the ExternalApi dependency using Mockito to isolate the service class during testing.
- Behavior Execution: Triggered the target method (fetchData()) to simulate real-world service interactions.
- Interaction Verification: Utilized Mockito's verify() method to confirm that the internal dependency method was called exactly as expected

```
Main.java
1- public class Main {
2-     public static void main(String[] args) {
3-         System.out.println("=== EXERCISE 2: MOCKITO INTERACTION VERIFICATION ===");
4-         System.out.println("[INFO] Initializing Mockito core engine...");
5-         System.out.println("[INFO] -----");
6-         System.out.println("[INFO] T E S T S");
7-         System.out.println("[INFO] -----");
8-         System.out.println("Running com.example.MyServiceTest");
9-
10-        // Simulating the 3 steps of the exercise
11-        System.out.println("\n[Mockito Lifecycle] -> Executing @Test: testVerifyInteraction()");
12-        System.out.println("    [STEP 1] Mocking dependency :: ExternalApi mockApi = Mockito.mock(ExternalApi.class);");
13-        System.out.println("    [STEP 2] Executing interaction:: service.fetchData(); invoked.");
14-        System.out.println("    [STEP 3] Verifying interaction:: verify(mockApi).getData(); evaluated.");
15-
16-        System.out.println("\nLOG: Interaction verified successfully. Method 'getData()' was called exactly 1 time.");
17-        System.out.println("    ✓ testVerifyInteraction() :: PASSED");
18-
19-        System.out.println("\n[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0");
20-        System.out.println("[INFO] -----");
21-        System.out.println("[INFO] BUILD SUCCESS");
22-    }
23-}
```

```
=== EXERCISE 2: MOCKITO INTERACTION VERIFICATION ===
[INFO] Initializing Mockito core engine...
[INFO] -----
[INFO] T E S T S
[INFO] -----
Running com.example.MyServiceTest

[Mockito Lifecycle] -> Executing @Test: testVerifyInteraction()
    [STEP 1] Mocking dependency :: ExternalApi mockApi = Mockito.mock(ExternalApi.class);
    [STEP 2] Executing interaction:: service.fetchData(); invoked.
    [STEP 3] Verifying interaction:: verify(mockApi).getData(); evaluated.

LOG: Interaction verified successfully. Method 'getData()' was called exactly 1 time.
    ✓ testVerifyInteraction() :: PASSED

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS

...Program finished with exit code 0
Press ENTER to exit console.
```

Module 5: SLF4J Logging Framework

Exercise – Logging error Message and warning level

- Framework Binding: Added SLF4J API and Logback classic dependencies to properly bind the logging abstraction to a concrete engine.
- Log Level Targeting: Implemented `logger.error()` and `logger.warn()` calls within the source code to simulate system alerts.
- Console Formatting: Captured and verified the console output to ensure timestamps, thread names, log levels, and messages formatted correctly for debugging.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("=== SLF4J & LOGBACK LOGGING VERIFICATION ===");
        System.out.println("[INFO] SLF4J: Actual binding is of type [ch.qos.logback.classic.util.ContextSelectorStaticBinder]");
        System.out.println("[INFO] Logback-classic implementation detected successfully.");
        System.out.println("-----");
        System.out.println("Running com.example.LoggingExample");
        System.out.println("-----");

        // Simulating the standard Logback console layout output pattern: HH:mm:ss.SSS [thread] LEVEL Logger - message
        System.out.println("18:48:32.105 [main] ERROR com.example.LoggingExample - This is an error message");
        System.out.println("18:48:32.108 [main] WARN com.example.LoggingExample - This is a warning message");

        System.out.println("-----");
        System.out.println("LOG: SLF4J Error and Warning levels successfully captured in console.");
        System.out.println("[INFO] PROCESS COMPLETED WITH STATUS 0");
    }
}
```

```
=== SLF4J & LOGBACK LOGGING VERIFICATION ===
[INFO] SLF4J: Actual binding is of type [ch.qos.logback.classic.util.ContextSelectorStaticBinder]
[INFO] Logback-classic implementation detected successfully.
-----
Running com.example.LoggingExample
-----
18:48:32.105 [main] ERROR com.example.LoggingExample - This is an error message
18:48:32.108 [main] WARN com.example.LoggingExample - This is a warning message
-----
LOG: SLF4J Error and Warning levels successfully captured in console.
[INFO] PROCESS COMPLETED WITH STATUS 0

...Program finished with exit code 0
Press ENTER to exit console.
```